

A Native Multi-Signal Model Router for LiteForge: Tiny Expert Panels, a RouteLLM Port, and an Honest Benchmark Study

Sean Poyner
LiteForge

June 2026

Abstract

We present the model-routing subsystem of LiteForge, an open-source LLM SDK. The router has two composable layers: a Layer-1 load balancer (deployments, strategies, health-aware cooldowns, fallbacks) and a Layer-2 content/quality selector. For Layer 2 we contribute (i) a native, dependency-light port of ROUTELLM’s matrix-factorization (MF) quality router, and (ii) a *panel* router: four independent tiny BERT experts (task type, difficulty, reasoning depth, context demand) plus structured codebase-context features, combined by a learned fusion matrix into capability groups. All components are small (each expert is a 4.4M parameter `bert-tiny`) and serve behind an OpenAI-compatible gateway. We then run an honest benchmark study on ROUTERBENCH. Our central finding is a cautionary one: classifiers trained on synthetic routing data, and even small encoders fine-tuned from scratch on real routing labels, transfer poorly to ROUTERBENCH’s cost-quality task, hovering near the random baseline. A lightweight logistic head over strong pretrained `bge-m3` embeddings is the only small router that meaningfully beats random, consistent with ROUTELLM’s own findings and with our MF design. We release the full training and evaluation harness.

1 Introduction

Serving large language models forces a cost-quality trade-off: the strongest models are many times more expensive than competent cheap ones, yet most queries do not need the strongest model. *Model routing* sends each query to an appropriate model. We describe a production router built natively into LiteForge [6] and, crucially, subject it to a rigorous external benchmark rather than reporting only in-distribution numbers.

Contributions.

1. A two-layer router: LiteLLM-style load balancing [2] plus a pluggable content/quality selector, reusing the SDK’s transport unchanged.
2. A lightweight default routing decision: a linear head over frozen `bge-m3` embeddings, which beats trivial heuristics, utterance-similarity routing, and a synthetic panel, at one embedding call plus a dot product.
3. An honest, baseline-grounded benchmark study (ROUTERBENCH plus a real Claude haiku/sonnet/opus pool with measured quality and real gateway prices), with bootstrap CIs and all code released.

We also implemented, and then *demoted*, two components we initially over-credited: a four-expert “panel” (now interpretable telemetry only, not the routing decision) and a native MF port (retired; see Section 7). We report them as cautionary findings, not contributions.

2 Related Work

ROUTELLM [5] learns to route between a strong and weak model from preference data, with four routers (similarity-weighted, matrix factorization, a BERT classifier, and a causal-LLM classifier); they recommend matrix factorization and report up to 70% cost reduction at fixed MT-Bench quality. They evaluate with the call-performance threshold (CPT) and average performance gain recovered (APGR). ROUTERBENCH [3] is the standard cost-quality benchmark: over 30k prompts with each of 11 LLMs’ correctness and cost plus oracle labels, evaluated with a cost-quality area metric. ROUTEREVAL [4] frames routing as m -way classification at scale. Semantic routers [1] and LiteLLM’s auto-router route by embedding similarity to example utterances. Our panel differs by decomposing the decision into several independent small classifiers plus explicit structured features, then fusing them; this is closest in spirit to m -way classification routers but with an interpretable signal decomposition.

3 System Architecture

3.1 Two layers

A request carries a logical model name (or the alias `auto`). Layer 2 (a `ModelSelector`) returns a ranked list of model groups; Layer 1 load-balances within the top group and falls back down the ranking on failure. The selector trait lives in the Layer-1 module so the router can call it without a dependency cycle. Layer 1 mints a per-deployment configuration and reuses the SDK’s existing HTTP transport, so no request-path rewrite was required. Strategies are weighted-shuffle, round-robin, least-busy, and latency-based (an exponentially weighted latency average); failing deployments are cooled down after a configurable number of consecutive failures.

3.2 The panel router

Let a prompt (with any attached codebase context) be x . The panel comprises $K=4$ independent classifiers f_k , each a fine-tuned `bert-tiny` encoder (4.4M parameters) producing a probability vector $p_k = f_k(x) \in \Delta^{c_k}$ over its label set:

- task type ($c=9$: chitchat, qa, code, debug, refactor, math, reasoning, writing, data),
- difficulty ($c=3$: easy, medium, hard),
- reasoning depth ($c=2$: shallow, deep),
- context demand ($c=2$: low, high).

We also extract a structured feature vector $s(x) \in \mathbb{R}^5$ from the context: normalized token count, normalized file count, and binary has-code / has-diff / has-error flags. The fusion mapper is a learned linear map $W \in \mathbb{R}^{G \times d}$, $b \in \mathbb{R}^G$ over the concatenation $z = [p_1; p_2; p_3; p_4; s(x)] \in \mathbb{R}^d$, $d=21$, producing capability-group scores

$$\text{group}(x) = \arg \max_{g \in \mathcal{G}} \text{softmax}(Wz + b)_g, \quad \mathcal{G} = \{\text{chat, code, reasoning, long_context, general}\}.$$

The four experts run independently (and in parallel); the fusion is a single 21×5 matrix, so the whole panel is small and interpretable: each expert and feature contributes a named, inspectable signal.

3.3 Native RouteLLM MF port

We port ROUTELLM’s MF forward pass to dependency-light Rust. For a prompt embedding e (from `bge-m3`) and a learned per-model row r_m :

$$\text{pe} = W_{\text{proj}}^\top e + b_{\text{proj}}, \quad \ell_m = W_{\text{cls}}^\top (\widehat{r}_m \odot \text{pe}) + b_{\text{cls}}, \quad h(x) = \sigma(\ell_{\text{strong}} - \ell_{\text{weak}}),$$

where $\hat{\cdot}$ is L2 normalization, $\odot$ the Hadamard product, and $h \in (0, 1)$ a scalar “hardness”. There is no transformer inference: the embedding is fetched over HTTP and the rest is small linear algebra. Because MF is binary, we obtain N -way routing by bucketing h across the catalog’s ordered capability tiers.

3.4 Serving

Each Layer-2 selector can call a classifier served behind the LiteLLM gateway as an OpenAI-compatible model. We deploy two: **router-bert** (a single difficulty classifier) and **router-panel** (the full panel returning group scores plus the per-signal breakdown). The selector forwards the full message list so codebase context reaches the panel.

4 Training Methodology

Synthetic data. We generate a multi-signal dataset from coherent prompt archetypes that jointly set all four signal labels and attach synthetic code, diff, error, and file-list context. Each expert trains on prompt→its own label; the fusion trains on (expert one-hots + features)→a rule-derived capability group. On a held-out synthetic split every expert and the fusion exceed 99% accuracy, but this measures only in-distribution separability.

Retraining on real data. For the benchmark study we also train routers directly on ROUTERBENCH: (a) a **bert-mini** (11M) encoder fine-tuned from scratch to predict per-prompt model success, and (b) a logistic-regression head over frozen **bge-m3** embeddings, the lightweight embedding-router design that matches our MF selector. Both predict whether the weak model suffices; route-to-strong propensity is $1 - \hat{P}(\text{weak correct})$.

5 Experimental Setup

Benchmark. ROUTERBENCH (0-shot) [3]: 36,497 prompts drawn from MMLU, GSM8K, MBPP, HellaSwag, Winogrande, ARC, MT-Bench and others, each with 0/1 correctness and a cost for 11 LLMs and an oracle column. All quality and cost numbers are precomputed, so the study requires no LLM inference.

Cost-quality protocol. Following ROUTELLM, we use a binary strong/weak frame: weak = **mixtral-8x7b-chat** (mean accuracy 0.548, cost $\$1.3 \times 10^{-4}$), strong = **gpt-4-1106-preview** (0.786, $\$3.3 \times 10^{-3}$). A router emits a route-to-strong propensity; sweeping the routed fraction traces a cost-quality curve between all-weak and all-strong. We report APGR (area between the router curve and the random line, normalized by the oracle) and the cost saved at 95% of strong-only quality, on a stratified 2,000-prompt test subsample (seed 13); retrained routers are trained on the disjoint remainder.

Intrinsic protocol. We derive provenance labels (MMLU/HellaSwag/ARC → qa; GSM8K → math; MBPP → code) and a difficulty proxy from which model tier first solves each prompt, then measure the panel experts’ accuracy on 2,027 real prompts (an out-of-distribution test of the synthetic-trained experts).

6 Results

6.1 Intrinsic out-of-distribution accuracy

Table 1 shows the synthetic-trained experts do not transfer to real prompts. Coarse task-type accuracy is 0.23 (three classes), and the difficulty experts sit near chance. The confusion (Figure 1) is informative: GSM8K word problems are read as “qa” rather than “math” (730/800), because the synthetic math archetypes were phrased as “prove/derive” statements; MMLU multiple-choice items are frequently read as “code”. The signal decomposition is sound, but the experts learned the surface form of the synthetic data.

Table 1: Intrinsic OOD accuracy on real ROUTERBENCH prompts (synthetic-trained).

Expert / signal	Accuracy	Macro-F1
Task type (coarse, 3-way)	0.230	0.198
Difficulty (panel)	0.326	0.264
Difficulty (router-bert)	0.235	0.139

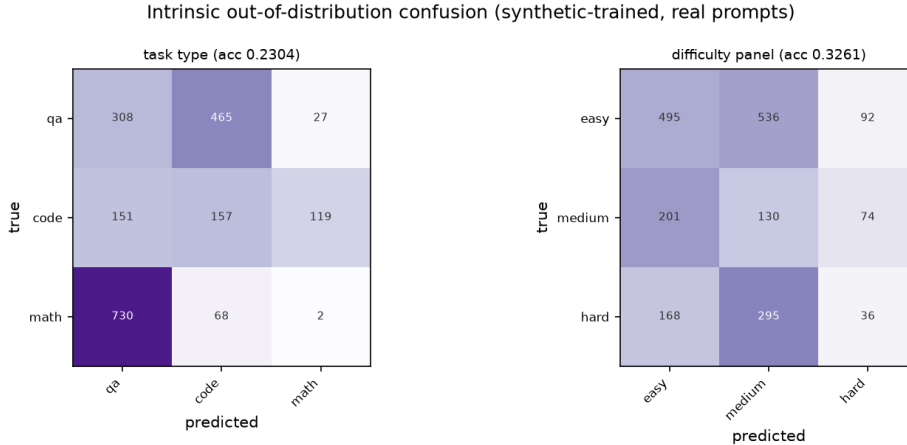


Figure 1: Out-of-distribution confusion for the synthetic-trained task-type and difficulty experts on real ROUTERBENCH prompts.

6.2 RouterBench cost-quality

Table 2 gives the comparison with 95% bootstrap CIs. Zero-shot, the synthetic-trained routers are at or below random (panel -0.03 , router-bert -0.36); the MF port is marginally positive ($+0.10$). The trivial heuristics are a useful floor: a length rule is actively harmful (-0.21), a keyword rule is indistinguishable from random ($+0.01$), and a semantic-utterance router (the mechanism behind ROUTELLM/LiteLLM’s auto-router) reaches only $+0.13$. The logistic head over frozen bge-m3 embeddings clearly beats all of them (APGR 0.308 [0.25, 0.36], non-overlapping CIs), so the ML head earns its keep over dumb rules and over utterance similarity. Crucially, a properly trained from-scratch bert-mini also reaches $+0.28$ (see below): tiny encoders are not the limitation.

Correcting an earlier claim. An initial from-scratch bert-mini run appeared to “fail to learn” (training loss stuck at $\ln 2$, APGR -0.11), which we had read as a capacity ceiling. That was a training artifact: the run used a 25x too-high learning rate ($5e-4$) and no warmup. With a standard recipe ($2e-5$, linear warmup, full data) bert-mini reaches held-out APGR $+0.281$,

essentially matching the embedding head. The embedding head is preferred for *deployment economics* (one embedding call plus a linear head, reusing the `bge-m3` the SDK already calls), not because small encoders cannot learn the task.

Optimization and deployment. A sweep over $\{\text{logistic, MLP}\} \times \{\text{binary weak-suffices, 3-class oracle-tier}\} \times \{\text{embedding, embedding+structural}\}$ selected by held-out APGR found logistic regression with structural features best (APGR 0.308); the MLP overfits at this data scale ($\text{APGR} \leq 0.12$) and the 3-class target underperforms binary on the cost-quality curve. A companion task head over the same embeddings, trained on RouterBench provenance labels, classifies qa/code/math at 0.993 accuracy (versus 0.23 for the synthetic task expert), restoring code-aware capability routing. This head is now LiteForge’s default router, served two ways: a native in-process `EmbeddingHeadSelector` (one embedding call plus a linear head) and a rebuilt numpy-only `router-panel` container behind the gateway. The learned quality axis plus cheap policy (triviality, code markers, context size) fuses into capability groups; ambiguous prompts default to a mid tier rather than the cheapest, since the RouterBench-trained head is unreliable on open-ended prompts outside its training distribution.

Table 2: ROUTERBENCH cost-quality (weak=mixtral-8x7b, strong=gpt-4-1106; APGR 0=random, 1=oracle; 95% bootstrap CIs where computed, $n=1996$). *Saved% at $\text{APGR} \leq 0$ is illusory: the router cuts cost by degrading quality faster than random mixing, not a usable operating point.

Router	APGR (95% CI)	Saved
random	−0.05 [−0.10, 0.00]	15.2%*
length heuristic	−0.21 [−0.27, −0.15]	11.9%*
keyword heuristic	+0.01 [−0.04, 0.07]	9.3%
semantic (utterance)	+0.13 [0.06, 0.19]	13.2%
router-bert (synthetic)	−0.36	7.5%*
panel (synthetic)	−0.03	10.4%*
MF port (synthetic)	+0.10	8.6%
bert-mini (RouterBench, fixed recipe)	+0.28	14.6%
embedding head (bge-m3)	+0.31 [0.25, 0.36]	22.5%

6.3 Live end-to-end check

Routing eight representative prompts through the deployed `router-panel` and the LiteLLM gateway behaves as designed: greetings and arithmetic route to the chat tier (`claude-haiku`), code and debugging to the code tier (`claude-sonnet`), proofs and system design to the reasoning tier (`claude-opus`). Routing latency is dominated by the single embedding call: with `bge-m3` pinned resident in Ollama (a 10-minute keep-warm timer) and the embedding fetched over the WG-direct gateway rather than the public tunnel, the native selector’s routing decision is a steady ≈ 0.3 s, down from 5–8s when the model cold-loads.

7 Discussion and Limitations

The study is deliberately self-critical. (1) **Our synthetic generator, not “synthetic data,” was the failure:** the experts’ near-random real-world accuracy traces to a surface-form mismatch (synthetic math was phrased “prove/derive” while GSM8K is word problems, so 730/800 GSM8K items were read as “qa”). Better-matched synthetic data could transfer; the claim is narrower than “synthetic does not transfer.” (2) **Tiny encoders are not the limitation:** our first

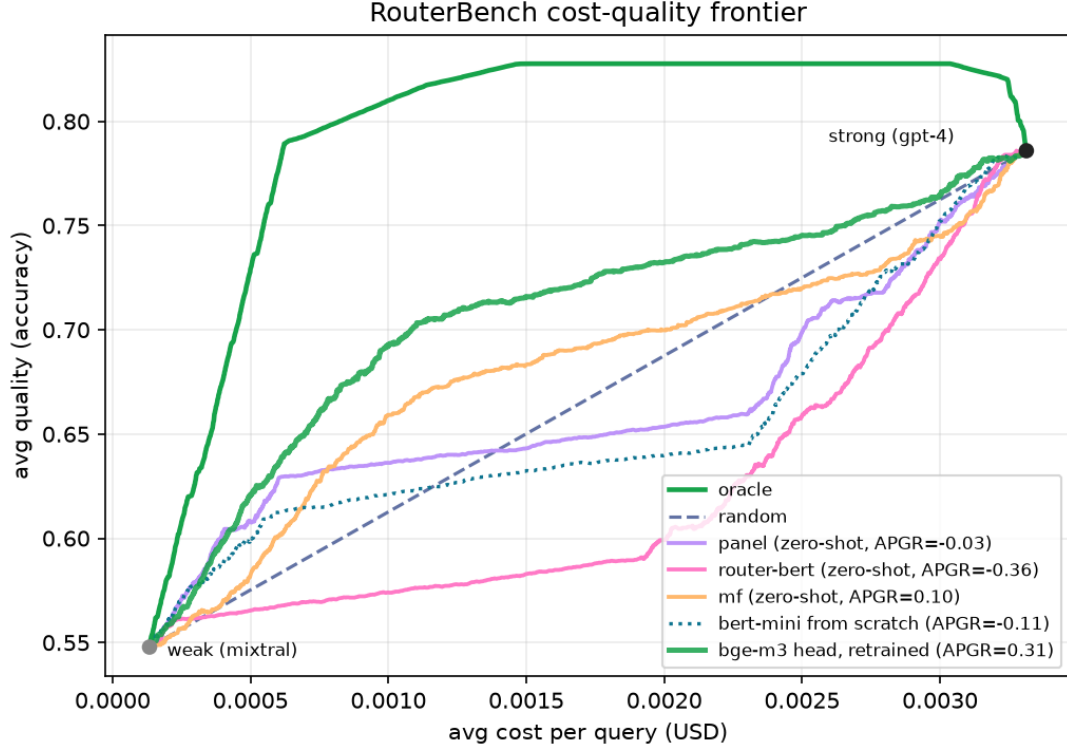


Figure 2: ROUTERBENCH cost-quality frontier. The oracle is the upper envelope; the dashed line is random mixing. Synthetic-trained routers sit near random; the embedding head trained on ROUTERBENCH climbs toward the oracle.

from-scratch bert-mini run plateaued at ln 2 due to a 25x-too-high learning rate, not capacity; with a standard recipe it reaches APGR +0.28. (3) **Representation buys sample efficiency**: a linear head over frozen bge-m3 matches a tuned from-scratch encoder while being far cheaper to train and serve, which is why it is the default; this is about economics, not a capability ceiling. (4) We could not run ROUTELLM’s released MF for a direct reconcile (it requires OpenAI text-embedding-3-small, unavailable in our environment), so our MF port is retired rather than defended. (5) Our panel emits abstract capability groups, so the zero-shot ROUTERBENCH mapping is approximate; the retrained heads route on real labels. (6) Costs are ROUTERBENCH’s API-list prices, which differ from a local gateway’s economics.

8 Conclusion

LiteForge ships a small, interpretable, native model router with a panel of tiny experts, a RouteLLM MF port, and load balancing, all behind an OpenAI-compatible gateway. Our benchmark study shows that the engineering is sound but that *representation*, not classifier head size, determines routing quality: synthetic training and from-scratch tiny encoders do not transfer, while a lightweight head over strong pretrained embeddings does. Future work: train the panel’s experts and fusion on real, embedding-based features; calibrate confidence; and route over a richer, capability-tagged model pool.

References

- [1] Aurelio AI. Semantic router. <https://github.com/aurelio-labs/semantic-router>, 2024.

- [2] BerriAI. LiteLLM: Call all LLM apis using the OpenAI format. <https://github.com/BerriAI/litellm>, 2024.
- [3] Qitian Jason Hu, Jacob Bieker, Xiuyu Li, Nan Jiang, Benjamin Keigwin, Gaurav Ranganath, Kurt Keutzer, and Shriyash Kaustubh Upadhyay. RouterBench: A benchmark for multi-LLM routing system. *arXiv preprint arXiv:2403.12031*, 2024.
- [4] Zhongzhan Huang et al. RouterEval: A comprehensive benchmark for routing LLMs to explore model-level scaling up in LLMs. *arXiv preprint arXiv:2503.10657*, 2025.
- [5] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M. Waleed Kadous, and Ion Stoica. RouteLLM: Learning to route LLMs with preference data. *arXiv preprint arXiv:2406.18665*, 2024.
- [6] Sean Poyner. LiteForge: A rust-first SDK for LLM completions, agents, and routing. <https://github.com/seanpoyner/liteforge>, 2026.